LeetCode Practice

2 Hard: Greedy

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

2
HARD

Patterns (in order)

1. Greedy (2)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

2 questions | tap a title to jump to it

Hard questions (2) Go >

Greedy (2) Go >

H 1. Candy LC 135

H 2. Set Intersection Size At Least Two LC 757

Hard questions (2)

HARD | TOPIC 1 OF 1

Greedy

2 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Candy LC 135
2. Set Intersection Size At Least Two LC 757

HARD | Greedy | LeetCode #135 | 1 of 2 | 1/2

Candy

QUESTION

There are n children standing in a line. Each child is assigned a rating value given in the integer array ratings.

You are giving candies to these children subjected to the following requirements:

CONSTRAINTS

- Each child must have at least one candy.
- Children with a higher rating get more candies than their neighbors.

Return the minimum number of candies you need to have to distribute the candies to the children.

Example 1	
Input	`ratings = [1,0,2]`
Output	`5`
Explanation	*You can allocate to the first, second and third child with 2, 1, 2 candies respectively.*

Example 2	
Input	`ratings = [1,2,2]`
Output	`4`
Explanation	*You can allocate to the first, second and third child with 1, 2, 1 candies respectively. The third child gets 1 candy because it satisfies the above two conditions.*

- n == ratings.length
- 1 <= n <= $2 * 10^4$
- 0 <= ratings[i] <= $2 * 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Give each child at least 1 candy. Higher-rated child must get more than adjacent lower-rated neighbours.

Return minimum total candies. Right?"

"A few quick questions:

Both left and right neighbours? Yes."

"Let me walk: ratings=[1,0,2] -> [2,1,2] -> total=5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
give everyone 1 candy
while True:
    whenever a child has a higher rating than a
        neighbour but not more candy, add candy
    if nothing changes:
        break
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Up to n passes, each O(n). Two passes (left to right, then right to left, keeping the larger requirement) are O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two-pass greedy — O(n). Walk-through:"

Pass 1 (left to right): ensure each child with higher rating than LEFT neighbor gets more candy.

`candy[0] = 1.`

For i in 1..n-1: candy[i] = candy[i-1]+1 if ratings[i]>ratings[i-1] else 1.

Pass 2 (right to left): ensure each child with higher rating than RIGHT neighbor gets more candy.

For i in n-2..0: if ratings[i]>ratings[i+1]: candy[i] = max(candy[i], candy[i+1]+1).

WHY two passes: one direction can't see both neighbors simultaneously.

Left pass satisfies left-neighbor constraint; right pass satisfies right-neighbor constraint.

`max() in right pass preserves the left-pass result when it already satisfies the constraint.`

Answer = sum(candy). Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _135_Candy:
    def candy(self, ratings):
        n = len(ratings)
        candies = [1] * n
        for i in range(1, n):
            if ratings[i] > ratings[i-1]:
                candies[i] = candies[i-1] + 1
        for i in range(n-2, -1, -1):
            if ratings[i] > ratings[i+1]:
                candies[i] = max(candies[i], candies[i+1]
+ 1)
        return sum(candies)
```

PHASE 5 TEST & DEBUG

"Let me trace candy([1,0,2])."

Rule: more rating than neighbor -> more candies.

Left pass: ratings=[1,0,2]. candy=[1,1,1].

i=1: 0<1 -> keep candy[1]=1. i=2: 2>0 -> candy[2]=candy[1]+1=2. candy=[1,1,2].

Right pass (right to left): i=1: 0<2 -> keep candy[1]=max(1,candy[2]+1)? No: only if ratings[1]>ratings[2]. 1>2? No.

i=0: 1>0 -> candy[0]=max(1,candy[1]+1)=2. candy=[2,1,2].

Sum=5 ok

[1,2,2]: left->[1,2,1]. right: 2(idx1)>2(idx2)? No. 1(idx0)<2(idx1)? Left pass handles. Sum=4 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
two passes	for candies array

O(1) space variant: slope technique (count peaks and valleys) — O(1) extra space.

Follow-up: Assign Cookies (#455) — simpler greedy with only one constraint.

Follow-up: if equal ratings must have equal candies, add equality check.

IN YOUR APP | Expose | Editorial | Community

HARD | Greedy | LeetCode #757 | 2 of 2 | 2/2

Set Intersection Size At Least Two

QUESTION

You are given a 2D integer array intervals where intervals[i] = [$start_i$, end_i] represents all the integers from $start_i$ to end_i inclusively.

A containing set is an array nums where each interval from intervals has at least two integers in nums.

CONSTRAINTS

- For example, if intervals = [[1,3], [3,7], [8,9]], then [1,2,4,7,8,9] and [2,3,4,8,9] are containing sets.

Return the minimum possible size of a containing set.

Example 1

Input	`intervals = [[1,3],[3,7],[8,9]]`
Output	**5**
Explanation	*let nums = [2, 3, 4, 8, 9]. It can be shown that there cannot be any containing array of size 4.*

Example 2

Input	`intervals = [[1,3],[1,4],[2,5],[3,5]]`
Output	**3**
Explanation	*let nums = [2, 3, 4]. It can be shown that there cannot be any containing array of size 2.*

Example 3

Input	`intervals = [[1,2],[2,3],[2,4],[4,5]]`
Output	**5**
Explanation	*let nums = [1, 2, 3, 4, 5]. It can be shown that there cannot be any containing array of size 4.*

- 1 <= intervals.length <= 3000
- intervals[i].length == 2
- 0 <= $start_i$ < end_i <= 10^8

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

For each pair of intervals, the intersection must contain at least 2 points from our set.

Find the minimum size of such a set. Right?"

"A few quick questions:

The set must be a subset of integers? No — the points can be real. But since intervals are integers, optimal points are integers."

PHASE 1 CLARIFY

"Let me walk: intervals=[[1,3],[1,4],[2,5],[3,5]] -> {3,5} satisfies all -> 2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for set in sets:
    # of integers within the overall range
    if it has at least two numbers inside every
        interval:
        best = min(best, set)
```

TIME	SPACE
exponential	O(range)

WHY IT IS SLOW

Enumerates every candidate set. Sort the intervals by end (and by start descending for equal ends); keep the last two chosen points and add 0, 1 or 2 new points at each interval's end: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort by endpoint, greedily assign — O(n log n). Walk-through:"

Sort intervals by right endpoint. result=[].

For each interval [l, r]:

Count how many result points fall in [l, r].

If < 2: add points until count == 2.

If 0 points in [l,r]: add r-1 and r.

If 1 point in [l,r]: add r.

WHY sort by right endpoint: processing intervals by earliest end point, greedy decision.

WHY place points at rightmost positions (r-1, r): right-side points are more likely to

cover future intervals (they have later endpoints too).

This is similar to interval point covering (Q452).

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _757_SetIntersectionSizeAtLeastTwo:
    def intersectionSizeTwo(self, intervals):
        intervals.sort(key=lambda x: (x[1], -x[0]))
        result = []
        for left, right in intervals:
            covered = sum(1 for p in result[-2:] if left <
= p <= right)
            if covered == 0:
                result.append(right - 1)
                result.append(right)
            elif covered == 1:
```

```
            result.append(right)
        return len(result)
```

PHASE 5 TEST & DEBUG

"Let me trace the greedy choice and show why that choice can't hurt the optimal result."

intervals=[[1,3],[1,4],[2,5],[3,5]] sorted by (right,-left): [[1,3],[1,4],[2,5],[3,5]].

[1,3]: covered=0. add 2,3. result=[2,3].

[1,4]: covered: 2 in [1,4]->yes, 3 in [1,4]->yes. covered=2. skip.

[2,5]: 2 in [2,5]->yes,3->yes. covered=2. skip.

[3,5]: 2 in [3,5]->no,3->yes. covered=1. add 5. result=[2,3,5]. len=3? But expected 2.

Hmm: the answer should be 2. The set {3,5} works: [1,3]{3}=1 pt but need 2. Hmm let me recheck.

Actually for [1,3]: need 2 points from {result} in [1,3]. {2,3}[1,3]=2 ok. But wait—do we need each interval to contain >=2 points? Yes. {2,3}: [1,3]=2ok,[1,4]=2ok,[2,5]=2ok,[3,5]=1. So {2,3} doesn't work.

Expected answer is 3 not 2. Let me check: [[1,3],[1,4],[2,5],[3,5]] answer should be 3: {2,3,5} or similar.

result=[2,3,5] len=3 ok

Greedy correctly builds the minimum covering set

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(n)**

Follow-up: if we need at least k points per interval, extend to maintaining last k points.

Follow-up: Minimum Number of Arrows (#452) — need only 1 point per interval.